

Suppose that when hedging your bets, you wait for p minutes, instead of for k minutes, before taking the stairs. What is the competitive ratio as a function of p and k ? How should you choose p to minimize the competitive ratio?

27.1-2

Imagine that you decide to take up downhill skiing. Suppose that a pair of skis costs r dollars to rent for a day and b dollars to buy, where $b > r$. If you knew in advance how many days you would ever ski, your decision whether to rent or buy would be easy. If you'll ski for at least $\lceil b/r \rceil$ days, then you should buy skis, and otherwise you should rent. This strategy minimizes the total that you ever spend. In reality, you don't know in advance how many days you'll eventually ski. Even after you have skied several times, you still don't know how many more times you'll ever ski. Yet you don't want to waste your money. Give and analyze an algorithm that has a competitive ratio of 2, that is, an algorithm guaranteeing that, no matter how many times you ski, you never spend more than twice what you would have spent if you knew from the outset how many times you'll ski.

27.1-3

In “concentration solitaire,” a game for one person, you have n pairs of matching cards. The backs of the cards are all the same, but the fronts contain pictures of animals. One pair has pictures of aardvarks, one pair has pictures of bears, one pair has pictures of camels, and so on. At the start of the game, the cards are all placed face down. In each round, you can turn two cards face up to reveal their pictures. If the pictures match, then you remove that pair from the game. If they don't match, then you turn both of them over, hiding their pictures once again. The game ends when you have removed all n pairs, and your score is how many rounds you needed to do so. Suppose that you can remember the picture on every card that you have seen. Give an algorithm to play concentration solitaire that has a competitive ratio of 2.

27.2 Maintaining a search list

The next example of an online algorithm pertains to maintaining the order of elements in a linked list, as in Section 10.2. This problem often arises in practice for hash tables when collisions are resolved by chaining (see Section 11.2), since each slot contains a linked list. Reordering the linked list of elements in each slot of the hash table can boost the performance of searches measurably.

The list-maintenance problem can be set up as follows. You are given a list L of n elements $\{x_1, x_2, \dots, x_n\}$. We'll assume that the list is doubly linked, although the algorithms and analysis work just as well for singly linked lists. Denote the position of element x_i in the list L by $r_L(x_i)$, where $1 \leq r_L(x_i) \leq n$. Calling LIST-SEARCH(L, x_i) on page 260 thus takes $\Theta(r_L(x_i))$ time.

If you know in advance something about the distribution of search requests, then it makes sense to arrange the list ahead of time to put the more frequently searched elements closer to the front, which minimizes the total cost (see Exercise 27.2-1). If instead you don't know anything about the search sequence, then no matter how you arrange the list, it is possible that every search is for whatever element appears at the tail of the list. The total searching time would then be $\Theta(nm)$, where m is the number of searches.

If you notice patterns in the access sequence or you observe differences in the frequencies in which elements are accessed, then you might want to rearrange the list as you perform searches. For example, if you discover that every search is for a particular element, you could move that element to the front of the list. In general, you could rearrange the list after each call to LIST-SEARCH. But how would you do so without knowing the future? After all, no matter how you move elements around, every search could be for the last element.

But it turns out that some search sequences are “easier” than others. Rather than just evaluate performance on the worst-case sequence, let's compare a reorganization scheme with whatever an optimal offline algorithm would do if it knew the search sequence in advance. That way, if the sequence is fundamentally hard, the optimal offline algorithm will also find it hard, but if the sequence is easy, we can hope to do reasonably well.

To ease analysis, we'll drop the asymptotic notation and say that the cost is just i to search for the i th element in the list. Let's also assume that the only way to reorder the elements in the list is by swapping two adjacent elements in the list. Because the list is doubly linked, each swap incurs a cost of 1. Thus, for example, a search for the sixth element followed by moving it forward two places (entailing two swaps) incurs a total cost 8. The goal is to minimize the total cost of calls to LIST-SEARCH plus the total number of swaps performed.

The online algorithm that we'll explore is MOVE-TO-FRONT(L, x). This procedure first searches for x in the doubly linked list L , and then it moves x to the front of the list.¹ If x is located at position $r = r_L(x)$ before the call, MOVE-TO-FRONT swaps x with the element in position $r - 1$, then with the element in position $r - 2$, and so on, until it finally swaps x with the element in position 1. Thus if the call MOVE-TO-FRONT($L, 8$) executes on the list $L = \langle 5, 3, 12, 4, 8, 9, 22 \rangle$, the list becomes $\langle 8, 5, 3, 12, 4, 9, 22 \rangle$. The call MOVE-TO-FRONT(L, k) costs $2r_L(k) - 1$: it costs $r_L(k)$ to search for k , and it costs 1 for each of the $r_L(k) - 1$ swaps that move k to the front of the list.

element searched	FORESEE					MOVE-TO-FRONT				
	L	search cost	swap cost	search + swap cost	cumulative cost	L	search cost	swap cost	search + swap cost	cumulative cost
5	$\langle 1, 2, 3, 4, 5 \rangle$	5	0	5	5	$\langle 1, 2, 3, 4, 5 \rangle$	5	4	9	9
3	$\langle 1, 2, 3, 4, 5 \rangle$	3	3	6	11	$\langle 5, 1, 2, 3, 4 \rangle$	4	3	7	16
4	$\langle 4, 1, 2, 3, 5 \rangle$	1	0	1	12	$\langle 3, 5, 1, 2, 4 \rangle$	5	4	9	25
4	$\langle 4, 1, 2, 3, 5 \rangle$	1	0	1	13	$\langle 4, 3, 5, 1, 2 \rangle$	1	0	1	26

Figure 27.1 The costs incurred by the procedures FORESEE and MOVE-TO-FRONT when searching for the elements 5, 3, 4, and 4, starting with the list $L = \langle 1, 2, 3, 4, 5 \rangle$. If FORESEE instead moved 3 to the front after the search for 5, the cumulative cost would not change, nor would the cumulative cost change if 4 moved to the second position after the search for 5.

We'll see that MOVE-TO-FRONT has a competitive ratio of 4. Let's think about what this means. MOVE-TO-FRONT performs a series of operations on a doubly linked list, accumulating cost. For comparison, suppose that there is an algorithm FORESEE that knows the future. Like MOVE-TO-FRONT, it also searches the list and moves elements around, but after each call it optimally rearranges the list for the future.

(There may be more than one optimal order.) Thus FORESEE and MOVE-TO-FRONT maintain different lists of the same elements.

Consider the example shown in Figure 27.1. Starting with the list $\langle 1, 2, 3, 4, 5 \rangle$, four searches occur, for the elements 5, 3, 4, and 4. The hypothetical procedure FORESEE, after searching for 3, moves 4 to the front of the list, knowing that a search for 4 is imminent. It thus incurs a swap cost of 3 upon its second call, after which no further swap costs accrue. MOVE-TO-FRONT incurs swap costs in each step, moving the found element to the front. In this example, MOVE-TO-FRONT has a higher cost in each step, but that is not necessarily always the case.

The key to proving the competitive bound is to show that at any point, the total cost of MOVE-TO-FRONT is not much higher than that of FORESEE. Surprisingly, we can determine a bound on the costs incurred by MOVE-TO-FRONT relative to FORESEE even though MOVE-TO-FRONT cannot see the future.

If we compare any particular step, MOVE-TO-FRONT and FORESEE may be operating on very different lists and do very different things. If we focus on the search for 4 above, we observe that FORESEE actually moves it to the front of the list early, paying to move the element to the front before it is accessed. To capture this concept, we use the idea of an ***inversion***: a pair of elements, say a and b , in which a appears before b in one list, but b appears before a in another list. For two lists L and L' , let $I(L, L')$, called the ***inversion count***, denote the number of inversions between the two lists, that is, the number of pairs of elements whose order differs in the two lists. For example, with lists $L = \langle 5, 3, 1, 4, 2 \rangle$ and $L' = \langle 3, 1, 2, 4, 5 \rangle$, then out of the $\binom{5}{2} = 10$ pairs, exactly five of them— $(1, 5)$, $(2, 4)$, $(2, 5)$, $(3, 5)$, $(4, 5)$ —are inversions, since these pairs, and only these pairs, appear in different orders in the two lists. Thus the inversion count is $I(L, L') = 5$.

In order to analyze the algorithm, we define the following notation. Let L_i^M be the list maintained by MOVE-TO-FRONT immediately after the i th search, and similarly, let L_i^F be FORESEE's list immediately after the i th search. Let c_i^M and c_i^F be the costs incurred by MOVE-TO-FRONT and FORESEE on their i th calls, respectively. We don't know how many swaps FORESEE performs in its i th call, but we'll denote

that number by t_i . Therefore, if the i th operation is a search for element x , then

$$c_i^M = 2r_{L_{i-1}^M}(x) - 1, \quad (27.3)$$

$$c_i^F = r_{L_{i-1}^F}(x) + t_i. \quad (27.4)$$

In order to compare these costs more carefully, let's break down the elements into subsets, depending on their positions in the two lists before the i th search, relative to the element x being searched for in the i th search. We define three sets:

$BB = \{\text{elements before } x \text{ in both } L_{i-1}^M \text{ and } L_{i-1}^F\},$

$BA = \{\text{elements before } x \text{ in } L_{i-1}^M \text{ but after } x \text{ in } L_{i-1}^F\},$

$AB = \{\text{elements after } x \text{ in } L_{i-1}^M \text{ but before } x \text{ in } L_{i-1}^F\}.$

We can now relate the position of element x in L_{i-1}^F and L_{i-1}^M to the sizes of these sets:

$$r_{L_{i-1}^M}(x) = |BB| + |BA| + 1, \quad (27.5)$$

$$r_{L_{i-1}^F}(x) = |BB| + |AB| + 1. \quad (27.6)$$

When a swap occurs in one of the lists, it changes the relative positions of the two elements involved, which in turn changes the inversion count. Suppose that elements x and y are swapped in some list. Then the only possible difference in the inversion count between this list and *any* other list depends on whether (x, y) is an inversion. In fact, the inversion count of (x, y) with respect to any other list *must* change. If (x, y) is an inversion before the swap, it no longer is afterward, and vice versa. Therefore, if two consecutive elements x and y swap positions in a list L , then for any other list L' , the value of the inversion count $I(L, L')$ either increases by 1 or decreases by 1.

As we compare MOVE-TO-FRONT and FORESEE searching and modifying their lists, we'll think about MOVE-TO-FRONT executing on its list for the i th time and then FORESEE executing on its list for the i th time. After MOVE-TO-FRONT has executed for the i th time and before FORESEE has executed for the i th time, we'll compare $I(L_{i-1}^M, L_{i-1}^F)$ (the inversion count immediately before the i th call of MOVE-TO-FRONT) with $I(L_i^M, L_{i-1}^F)$ (the inversion count after the i th call of

MOVE-TO-FRONT but before the i th call of FORESEE). We'll concern ourselves later with what FORESEE does.

Let us analyze what happens to the inversion count after executing the i th call of MOVE-TO-FRONT, and suppose that it searches for element x . More precisely, we'll compute $I(L_i^M, L_{i-1}^F) - I(L_{i-1}^M, L_{i-1}^F)$, the change in the inversion count, which gives a rough idea of how much MOVE-TO-FRONT's list becomes more or less like FORESEE's list. After searching, MOVE-TO-FRONT performs a series of swaps with each of the elements on the list L_{i-1}^M that precedes x . Using the notation above, the number of such swaps is $|BB| + |BA|$. Bearing in mind that the list L_{i-1}^F has yet to be changed by the i th call of FORESEE, let's see how the inversion count changes.

Consider a swap with an element $y \in BB$. Before the swap, y precedes x in both L_{i-1}^M and L_{i-1}^F . After the swap, x precedes y in L_i^M , and L_{i-1}^F does not change. Therefore, the inversion count increases by 1 for each element in BB . Now consider a swap with an element $z \in BA$. Before the swap, z precedes x in L_{i-1}^M but x precedes z in L_{i-1}^F . After the swap, x precedes z in both lists. Therefore, the inversion count decreases by 1 for each element in BA . Thus altogether, the inversion count increases by

$$I(L_i^M, L_{i-1}^F) - I(L_{i-1}^M, L_{i-1}^F) = |BB| - |BA| . \quad (27.7)$$

We have laid the groundwork needed to analyze MOVE-TO-FRONT.

Theorem 27.1

Algorithm MOVE-TO-FRONT has a competitive ratio of 4.

Proof The proof uses a potential function, as described in Chapter 16 on amortized analysis. The value Φ_i of the potential function after the i th calls of MOVE-TO-FRONT and FORESEE depends on the inversion count:

$$\Phi_i = 2I(L_i^M, L_i^F) .$$

(Intuitively, the factor of 2 embodies the notion that each inversion represents a cost of 2 for MOVE-TO-FRONT relative to FORESEE: 1

for searching and 1 for swapping.) By equation (27.7), after the i th call of MOVE-TO-FRONT, but before the i th call of FORESEE, the potential increases by $2(|BB| - |BA|)$. Since the inversion count of the two lists is nonnegative, we have $\Phi_i \geq 0$ for all $i \geq 0$. Assuming that MOVE-TO-FRONT and FORESEE start with the same list, the initial potential Φ_0 is 0, so that $\Phi_i \geq \Phi_0$ for all i .

Drawing from equation (16.2) on page 456, the amortized cost $\hat{c}_i^M$ of the i th MOVE-TO-FRONT operation is

$$\hat{c}_i^M = c_i^M + \Phi_i - \Phi_{i-1},$$

where c_i^M , the actual cost of the i th MOVE-TO-FRONT operation, is given by equation (27.3):

$$c_i^M = 2r_{L_{i-1}^M}(x) - 1.$$

Now, let's consider the potential change $\Phi_i - \Phi_{i-1}$. Since both L^M and L^F change, let's consider the changes to one list at a time. Recall that when MOVE-TO-FRONT moves element x to the front, it increases the potential by exactly $2(|BB| - |BA|)$. We now consider how the optimal algorithm FORESEE changes its list L^F : it performs t_i swaps. Each swap performed by FORESEE either increases or decreases the potential by 2, and thus the increase in potential by FORESEE in the i th call can be at most $2t_i$. We therefore have

$$\begin{aligned} \hat{c}_i^M &= c_i^M + \Phi_i - \Phi_{i-1} \\ &\leq 2r_{L_{i-1}^M}(x) - 1 + 2(|BB| - |BA| + t_i) \\ &= 2r_{L_{i-1}^M}(x) - 1 + 2(|BB| - (r_{L_{i-1}^M}(x) - 1 - |BB|) + t_i) \\ &\quad \text{(by equation (27.5))} \\ &= 4|BB| + 1 + 2t_i \\ &\leq 4|BB| + 4|AB| + 4 + 4t_i && \text{(increasing some terms)} \\ &= 4(|BB| + |AB| + 1 + t_i) \\ &= 4(r_{L_{i-1}^F}(x) + t_i) && \text{(by equation (27.6))} \\ &= 4c_i^F && \text{(by equation (27.4))} . \end{aligned} \tag{27.8}$$

We now finish the proof as in Chapter 16 by showing that the total amortized cost provides an upper bound on the total actual cost, because the initial potential function is 0 and the potential function is

always nonnegative. By equation (16.3) on page 456, for any sequence of m MOVE-TO-FRONT operations, we have

$$\begin{aligned}\sum_{i=1}^m \hat{c}_i^M &= \sum_{i=1}^m c_i^M + \Phi_m - \Phi_0 \\ &\geq \sum_{i=1}^m c_i^M \quad (\text{because } \Phi_m \geq \Phi_0) .\end{aligned}\tag{27.9}$$

Therefore, we have

$$\begin{aligned}\sum_{i=1}^m c_i^M &\leq \sum_{i=1}^m \hat{c}_i^M \quad (\text{by equation (27.9)}) \\ &\leq \sum_{i=1}^m 4c_i^F \quad (\text{by equation (27.8)}) \\ &= 4 \sum_{i=1}^m c_i^F .\end{aligned}$$

Thus the total cost of the m MOVE-TO-FRONT operations is at most 4 times the total cost of the m FORESEE operations, so MOVE-TO-FRONT is 4-competitive. ■

Isn't it amazing that we can compare MOVE-TO-FRONT with the optimal algorithm FORESEE when we have no idea of the swaps that FORESEE makes? We were able to relate the performance of MOVE-TO-FRONT to the optimal algorithm by capturing how particular properties (swaps in this case) must evolve relative to the optimal algorithm, without actually knowing the optimal algorithm.

The online algorithm MOVE-TO-FRONT has a competitive ratio of 4: on any input sequence, it incurs a cost at most 4 times that of any other algorithm. On a particular input sequence, it could cost much less than 4 times the optimal algorithm, perhaps even matching the optimal algorithm.

Exercises

27.2-1

You are given a set $S = \{x_1, x_2, \dots, x_n\}$ of n elements, and you wish to make a static list L (no rearranging once the list is created) containing

the elements of S that is good for searching. Suppose that you have a probability distribution, where $p(x_i)$ is the probability that a given search searches for element x_i . Argue that the expected cost for m searches is

$$m \sum_{i=1}^n p(x_i) \cdot r_L(x_i) .$$

Prove that this sum is minimized when the elements of L are sorted in decreasing order with respect to $p(x_i)$.

27.2-2

Professor Carnac claims that since FORESEE is an optimal algorithm that knows the future, then at each step it must incur no more cost than MOVE-TO-FRONT. Either prove that Professor Carnac is correct or provide a counterexample.

27.2-3

Another way to maintain a linked list for efficient searching is for each element to maintain a *frequency count*: the number of times that the element has been searched for. The idea is to rearrange list elements after searches so that the list is always sorted by decreasing frequency count, from largest to smallest. Either show that this algorithm is $O(1)$ -competitive, or prove that it is not.

27.2-4

The model in this section charged a cost of 1 for each swap. We can consider an alternative cost model in which, after accessing x , you can move x anywhere earlier in the list, and there is no cost for doing so. The only cost is the cost of the actual accesses. Show that MOVE-TO-FRONT is 2-competitive in this cost model, assuming that the number requests is sufficiently large. (*Hint*: Use the potential function $\Phi_i = I(L_i^M, L_i^F)$.)

27.3 Online caching